Intelligent Project Management System for Undergraduate Students

INDIVIDUAL THESIS

Bandara N D M

IT22114662

Individual component: Intelligent Team Formation and Topic Feasibility Analysis

Group ID: R26-IT-095

Report submitted in partial fulfilment of the requirements for the

Information Technology

Faculty of Computing

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

2026 October

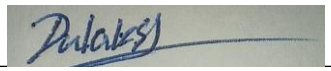
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure**.** I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Bandara N D M

Student ID: IT22114662

Signature: _____  ____ Date: _____10/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Prof. Samantha Rajapaksha		
Ms. Kaushika Kahatapitiya		

ABSTRACT

Undergraduate project team formation requires balancing students' technical competencies with their preferred project topics. Existing approaches often emphasize student characteristics or combine multiple criteria into a single weighted score, which can obscure the trade-off between technical project suitability and student preference. This research presents an intelligent multi-objective framework for undergraduate team and project allocation using Technical Requirement Deficit and Preference Dissatisfaction as separate objectives. Student competencies are represented using self-reported ordinal ratings, while project requirements specify technologies, minimum competency thresholds, and required numbers of qualified members.

A Heuristic-Seeded NSGA-II V3 approach was developed to generate nondominated allocation alternatives for academic staff. The method combines technical-greedy and preference-greedy initialization with local variants and random feasible solutions. It also supports surplus approved projects, remainder teams, technical coverage inspection, supervisor allocation, final allocation persistence, and REST based integration with the wider Intelligent Project Management System.

Evaluation included exact Pareto verification, controlled baseline comparison, statistical testing, scalability analysis, structural constraint testing, and a real-response case study. For a 12 student controlled problem, the method recovered the complete exact Pareto fronts in the verified medium and high conflict cases. Across the controlled evaluation, it achieved the highest mean hypervolume of 0.9479, approximately 45.1% higher than unseeded NSGA-II. A synthetic 1,000 student, 250 project scenario was processed in approximately 30.15 seconds. In the 60 student real response case study, the framework achieved 100% modeled technical requirement coverage and 95% preference satisfaction.

The findings indicate that project aware multi-objective optimization can provide transparent and scalable decision support while preserving academic staff authority over final allocation decisions.

Keywords: *Multi-objective optimization, Team formation, NSGA-II, Project allocation, Decision support*

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Prof. Samantha Rajapaksha**, and co-supervisor, **Ms. Kaushika Kahatapitiya**, for their valuable guidance, constructive feedback, and continuous support throughout this research project. Their advice was important in refining the research problem, methodology, evaluation process, and development of the Intelligent Team Formation and Topic Feasibility Analysis component.

I also extend my appreciation to the academic staff of the Sri Lanka Institute of Information Technology for providing the knowledge, resources, and research environment required to complete this work.

I am grateful to the students who participated in the data collection process and provided the competency and project-preference information used in the real-response case study. Their participation contributed directly to the practical evaluation of the proposed framework.

I would also like to thank my research group members for their cooperation in developing and integrating the complete **Intelligent Project Management System for Undergraduate Students**. Their collaboration was important in ensuring that the individual components could operate together as an integrated solution.

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1. Overall project context 1
1.2. Individual component context 1
1.3. Problem statement 2
1.4. Research gap 3
1.5. Aim and objectives 3
1.6. Research questions or hypotheses 4
1.7. Scope limitations and assumptions 5
1.8. Individual contribution 5
1.9. Thesis structure 6
2. LITERATURE REVIEW AND RELATED WORK 7
2.1. Review method and scope 7
2.2. Domain and theoretical foundations 7
2.3. Related research 8
2.3.1. Educational Team Formation 8
2.3.2. Competency-Based Software Staffing 8
2.3.3. Evolutionary Team Formation 9
2.4. Existing systems products and techniques 9
2.5. Gap synthesis and conceptual framework 12
3. RESEARCH METHODOLOGY 14
3.1. Research design 14
3.2. Data participants or study materials 14
3.3. Experimental or study procedure 15
3.3.1. Problem Representation 15

3.3.2. Technical Requirement Deficit 16
3.3.3. Preference Dissatisfaction 16
3.3.4. Heuristic-Seeded NSGA-II V3 17
3.3.5. Surplus Approved Projects 18
3.4. Measures and evaluation metrics 18
3.5. Data analysis 19
3.6. Ethics, privacy, security and responsible AI 20
3.7. Reproducibility and research artefacts 20
4. PRODUCT AND SOLUTION DESIGN 22
4.1. Requirements 22
4.2. Component architecture within the complete solution 23
4.3. Design alternatives and decisions 24
4.4. Detailed design 25
4.5. Standards and professional practice 27
5. IMPLEMENTATION INTEGRATION AND TESTING 29
5.1. Development environment and technology stack 29
5.2. Component implementation 30
5.3. Integration with the other components 31
5.4. Verification and software testing 32
5.5. Deployment and operation 33
6. RESULTS AND VALIDATION 36
6.1. Research results 36
6.1.1 Exact Pareto Verification 36
6.1.2 Baseline Comparison 36
6.1.3 Scalability Results 38
6.1.4 Surplus Project Constraint Results 38

6.1.5 Real Response Case Study 39
6.2. Product and technical results 40
6.3. User or stakeholder validation 41
6.4. Integration results 42
6.5. Summary of findings 42
7. DISCUSSION 44
7.1. Interpretation of findings 44
7.2. Comparison with literature baselines and existing solutions 45
7.3. Research contribution 46
7.4. Product contribution 46
7.5. Limitations and threats to validity 47
7.6. Implications 48
8. CONCLUSION AND FUTURE DIRECTIONS 50
8.1. Conclusion 50
8.2. Achievement of objectives 50
8.3. Individual research and product contributions 51
8.4. Future directions 52
REFERENCES 54
APPENDICES 56
Appendix A Research instruments or extended results 56
Appendix B Academic integrity and AI use disclosure 58

LIST OF FIGURES

FIGURE 2.1 CONCEPTUAL FRAMEWORK OF THE PROPOSED TEAM-PROJECT ALLOCATION APPROACH 13

FIGURE 4.1 ARCHITECTURE OF THE INTELLIGENT TEAM FORMATION AND TOPIC FEASIBILITY COMPONENT 24

FIGURE 4.2 STAFF WORKFLOW OF THE TEAM-FORMATION COMPONENT ..... ……26

FIGURE 5.1 INTEGRATION OF THE TEAM FORMATION COMPONENT WITH IPMS 32

FIGURE 6.1 OVERALL MEAN HYPERVOLUME OF THE EVALUATED ALLOCATION METHODS 37

LIST OF TABLES

TABLE 2.1 POSITIONING RELATIVE TO REPRESENTATIVE PRIOR WORK 10

TABLE 4.1 FUNCTIONAL REQUIREMENTS OF THE TEAM FORMATION COMPONENT 23

TABLE 5.1 MAIN TECHNOLOGY STACK 29

TABLE 6.1 OVERALL MEAN HYPERVOLUME BY METHOD …….....36

TABLE 6.2 SURPLUS PROJECT CONSTRAINT VERIFICATION …….....38

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DEAP	Distributed Evolutionary Algorithms in Python
GA	Genetic Algorithm
HTTP	Hypertext Transfer Protocol
IPMS	Intelligent Project Management System
JSON	JavaScript Object Notation
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PDF	Portable Document Format
REST	Representational State Transfer
RQ	Research Question
UI	User Interface
URL	Uniform Resource Locator

1. INTRODUCTION

1.1. Overall project context

Undergraduate research projects involve several connected activities, including team formation, project allocation, project planning, proposal preparation, progress monitoring, and contribution assessment. These activities are often managed separately, which can increase the workload placed on academic staff and make it difficult to maintain consistency throughout the project lifecycle.

The Intelligent Project Management System for Undergraduate Students (IPMS) was developed to provide integrated decision support for these activities. The complete system contains four research components: intelligent team formation and topic feasibility analysis, generative project planning and adaptive scheduling, AI based research proposal analysis and improvement, and predictive risk monitoring with contribution analytics.

The overall system is designed to assist academic staff rather than replace academic decision making. Each component generates information, recommendations, or alternatives that can be reviewed before a final decision is made. This thesis focuses on the first component, which addresses undergraduate team formation and project allocation.

1.2. Individual component context

The individual component of this research is **Intelligent Team Formation and Topic Feasibility Analysis**. Its purpose is to support academic staff when forming student teams and assigning approved projects.

The allocation process considers two main factors. The first is whether a team collectively satisfies the technical requirements of its assigned project. The second is whether the assigned project matches the ranked preferences of the students. These factors may conflict because an allocation that provides strong technical coverage

may not provide the highest preference satisfaction, while a preference-focused allocation may leave technical gaps.

Student competency is represented using self-reported ordinal ratings from 1 to 5 across a defined set of technologies. Projects contain explicit technical requirements specifying the technology, minimum competency level, and required number of qualified members. The framework then evaluates **Technical Requirement Deficit** $\boldsymbol{f_1}$ and **Preference Dissatisfaction** $\boldsymbol{f_2}$ as separate objectives.

A Heuristic-Seeded NSGA-II approach is used to generate nondominated allocation alternatives. Academic staff can compare these alternatives and select an appropriate allocation rather than being given a single automatically selected result.

1.3. Problem statement

Undergraduate team formation is more complex than dividing students into groups of equal size because teams must also be matched with suitable projects. Different projects require different technologies and levels of technical competency, while students may have different project preferences.

The two requirements can conflict. Prioritizing technical suitability may reduce student preference satisfaction, whereas prioritizing student preference may leave some project requirements insufficiently covered. The number of possible allocations also increases rapidly as the cohort grows. For example, a case with only 12 students, three projects, and teams of four already contains 34,650 feasible complete allocations**.**

Another difficulty occurs when the number of approved projects is greater than the number of required student teams. In such cases, the system must determine which projects should be considered in the final allocation as well as how the students should be grouped.

Therefore, the research problem is the absence of a practical decision support approach that considers both explicit project technical requirements and student

preferences while producing multiple feasible allocation alternatives for academic review.

1.4. Research gap

Educational team formation research has considered factors such as cohesion, personality, student characteristics, and learning activity [1]-[4]. These studies demonstrate the usefulness of computational team formation but generally focus on the composition of the student team rather than the technical requirements of alternative projects.

Software-engineering staffing research has considered competencies, project requirements, roles, and multi-objective optimization [5]-[8]. These approaches provide stronger support for competency-requirement matching but are primarily designed for professional staffing environments rather than undergraduate project allocation.

Other studies have used genetic algorithms and Pareto-based approaches for group and team formation [9], [10]. However, limited attention has been given to undergraduate project allocation where project technical suitability and student project preference are retained as separate competing objectives and presented as nondominated alternatives for academic staff.

The research therefore addresses this gap using an interpretable project-requirement representation, separate technical and preference objectives, heuristic-seeded multi-objective optimization, and a staff-oriented decision-support process. NSGA-II itself is an established algorithm [11]; the contribution lies in how it is adapted to this particular undergraduate project allocation problem.

1.5. Aim and objectives

The aim of this research is:

To develop and evaluate an intelligent multi-objective framework for undergraduate project team formation and project allocation that considers project technical requirements and student preferences while supporting academic decision making.

The specific objectives are:

- To represent student competency and project technical requirements in an interpretable form.
- To formulate technical requirement deficit and preference dissatisfaction as separate optimization objectives.
- To develop a Heuristic-Seeded NSGA-II approach for generating feasible nondominated allocations.
- To support cases where approved projects exceed the number of required student teams.
- To evaluate the proposed approach using controlled experiments, baseline comparisons, scalability testing, and a real-response case study.

1.6. Research questions or hypotheses

The study addresses the following research questions:

RQ1: How can student competencies be matched to explicit project technical requirements in an interpretable form for undergraduate project team formation?

RQ2: How effectively can a multi objective formulation expose tradeoffs between technical suitability and student project preference?

RQ3: How does Heuristic-Seeded NSGA-II compare with greedy, random, weighted single-objective, and unseeded optimization approaches?

These questions cover the representation of the problem, the usefulness of the multi-objective formulation, and the performance of the proposed optimization approach.

1.7. Scope limitations and assumptions

The research focuses on undergraduate project team formation and project allocation within IPMS. Student competency is represented using self-reported ratings from 1 to 5 across 16 technologies. These ratings are treated as competency indicators and are not considered objective measurements of technical ability.

Projects are represented using explicit technical requirements consisting of a technology, a minimum competency threshold, and a required number of qualified team members. Students also provide ranked project preferences, which are used to calculate preference dissatisfaction.

The method supports a target team size and allows a smaller remainder team when the number of students is not exactly divisible by that size. When approved projects outnumber required teams, some projects may remain unused. When the number of required teams exceeds the number of available unique projects, the allocation is considered infeasible.

The value of f_1 indicates full coverage of the modeled technical requirements only. It does not guarantee project success, academic performance, or objectively verified technical competence. Similarly, preference satisfaction reflects ranked project choices rather than overall student satisfaction.

The 60 student case study represents one real response dataset, while the 1,000 student scalability experiment uses synthetic data. These results should therefore be interpreted within their respective evaluation purposes.

1.8. Individual contribution

The individual research contribution consists of four main elements.

First, an interpretable project-requirement representation was developed using technology, minimum competency level, and required number of qualified members.

Second, the team-project allocation problem was formulated using two independent objectives: Technical Requirement Deficit and Preference Dissatisfaction. Keeping these objectives separate allows tradeoffs to remain visible.

Third, a Heuristic-Seeded NSGA-II V3 approach was developed using technical oriented, preference-oriented, local variation, and random initialization strategies. The contribution is the problem-specific formulation and adaptation, not the invention of NSGA-II itself.

Fourth, the optimization method was integrated into a staff oriented workflow that allows academic users to compare nondominated alternatives, inspect technical coverage, select an allocation, and proceed to supervisor allocation and finalization.

The research was evaluated using exact Pareto verification, baseline comparisons, statistical testing, scalability experiments, structural regression testing, and a real-response case study.

1.9. Thesis structure

Chapter 2 reviews literature related to educational team formation, competency-based personnel allocation, and multi-objective optimization. Chapter 3 presents the research methodology, including the objective formulation, optimization process, datasets, and evaluation methods. Chapter 4 describes the product and solution design, while Chapter 5 explains implementation, integration, and testing. Chapter 6 presents the research and technical results. Chapter 7 discusses the findings, contributions, and limitations. Finally, Chapter 8 concludes the thesis and presents future research directions.

2. LITERATURE REVIEW AND RELATED WORK

2.1. Review method and scope

This review focuses on research relevant to undergraduate project-team formation and project allocation. The literature was examined across three areas: educational team formation, software engineering personnel allocation, and evolutionary or multi-objective optimization.

Educational studies were considered because the research is conducted in an undergraduate environment. Professional software staffing research was reviewed because it provides approaches for matching competencies to project requirements. Optimization studies were examined because the allocation problem contains a large search space and competing objectives.

The review is therefore limited to work that directly supports the formulation of the proposed team project allocation framework rather than attempting to cover every possible factor associated with teamwork.

2.2. Domain and theoretical foundations

Team formation involves assigning individuals to groups according to one or more desired criteria. In educational environments, these criteria may include student characteristics, personality, learning behavior, cohesion, or competency. In project-based education, however, the team must also be matched with an appropriate project.

This introduces a competency requirement relationship. A team may appear balanced generally but may still be unsuitable for a project requiring technologies that are not sufficiently represented among its members. For this reason, both student information and project requirements must be represented.

The second foundation of this research is multi objective optimization. Technical suitability and student project preference may conflict, meaning that improving one objective can worsen the other. Combining them into one weighted score requires predetermined importance values.

Pareto-based optimization avoids this requirement by maintaining multiple nondominated solutions. A solution is nondominated when no other solution improves one objective without worsening at least one other objective.

NSGA-II [11] is a widely used multi objective evolutionary algorithm that applies nondominated sorting and diversity preservation. In this research, NSGA-II provides the optimization foundation, while the project representation, objectives, heuristic seeding, and workflow are specific to the proposed problem.

2.3. Related research

2.3.1. Educational Team Formation

Educational team-formation research has considered several student related characteristics. Shaikh [1] identified criteria associated with cohesive software-engineering capstone teams. Løvold et al. [2] examined student-selected and instructor-assigned teams. Sanchez-Anguix et al. [3] studied computational grouping based on personality and team-role characteristics, while Liang et al. [4] applied a mixed genetic algorithm using authentic learning activity data.

These studies demonstrate that educational team formation can be supported computationally. However, their primary focus is generally on the composition of the student team rather than the explicit technical requirements of alternative projects.

2.3.2. Competency-Based Software Staffing

Requirement-aware assignment is more common in professional software engineering. André et al. [5] considered competencies, roles, availability, and project constraints when assigning personnel to software project teams.

Paredes-Valverde et al. [6] used an ontology-based method for matching personnel competencies with project requirements. These studies show the importance of representing project needs explicitly rather than assessing personnel independently from the project.

Stylianou and Andreou [7] used multi-objective genetic optimization for software team staffing using technical and personality-related factors. Arunachalam et al. [8] applied NSGA-II to agile team selection using cost and productivity objectives.

These approaches provide important foundations for competency-aware and multi-objective allocation, although their professional staffing objectives differ from the undergraduate project context.

2.3.3. Evolutionary Team Formation

Genetic algorithms are useful for team formation because the number of possible group combinations increases rapidly with cohort size.

Moreno et al. [9] used a genetic algorithm to form collaborative-learning groups based on multiple student characteristics. Casotti and Krohling [10] investigated a bi-objective team-formation problem using Pareto-based evolutionary algorithms and hypervolume evaluation.

These studies demonstrate that evolutionary algorithms can search complex team-formation spaces and that Pareto-based approaches are useful when multiple objectives must be considered simultaneously.

The proposed work builds on these ideas but focuses specifically on the combination of explicit project technical requirements and ranked student project preferences.

2.4. Existing systems products and techniques

Existing research uses several techniques for team and personnel allocation, including manual or instructor-led grouping, criteria-based grouping, heuristic methods, competency matching, genetic algorithms, and multi-objective optimization.

Manual or instructor-led grouping provides academic control but becomes difficult when cohorts and project pools are large. Student self-selection can reflect student interests but does not guarantee suitable technical coverage.

Competency requirement matching improves project suitability by explicitly connecting personnel skills with project needs [5], [6]. However, professional staffing models do not directly address ranked undergraduate project preferences.

Single-objective genetic algorithms can search large allocation spaces efficiently, but combining technical suitability and preference into one fitness score requires a fixed weighting. Multi-objective algorithms such as NSGA-II avoid this by returning nondominated alternatives.

The proposed framework combines competency-requirement matching with multi-objective optimization and academic staff review. It also supports cases where approved projects outnumber required teams, which require project pool selection in addition to student grouping.

TABLE 2.1: Positioning Relative to Representative Prior Work

Study	**Context**	**Main Focus**	**Approach**	**Relevance / Distinction**
Shaikh [1]	Education	Cohesive capstone-team criteria	Criteria based	Broad team factors; not project requirement centered
Løvold et al. [2]	Education	Student selected vs instructor assigned teams	Empirical educational comparison	Shows different levels of student/staff control
Sanchez-Anguix et al. [3]	Education	Personality and team-role grouping	Computational heuristics	Algorithmic grouping based on student attributes
Liang et al.	Education	Learning	Mixed genetic	Authentic

[4]		activity data	algorithm	educational optimization
André et al. [5]	Professional software	Competencies, roles, availability and constraints	Formal model	Strong project/personnel requirement representation
Paredes-Valverde et al. [6]	Professional software	Competency-requirement matching	Ontology-based	Structured matching of personnel to project needs
Stylianou and Andreou [7]	Professional software	Technical skills and personality	Multi-objective GA	Demonstrates multiple staffing criteria
Arunachalam et al. [8]	Professional software	Agile team selections, cost and productivity	NSGA-II	Pareto based team selection precedent
Moreno et al. [9]	Education	Multiple student characteristics	Genetic algorithm	Evolutionary educational group formation
Casotti and Krohling [10]	Team Formation	Bi-objective team formation	Pareto-based evolutionary algorithms	Relevant to nondominated set and hypervolume evaluation
Proposed research	**Undergraduate Projects**	**Explicit project technical**	**Heuristic-Seeded NSGA-II +**	**Generates staff selectable non dominated**

		requirements + ranked project preference	**Project subnet handling**	**team-project allocations**

Table 2.1 shows that educational studies primarily focus on student or team characteristics, while professional staffing studies provide stronger project requirement models. The proposed research combines elements from both areas.

2.5. Gap synthesis and conceptual framework

The literature identifies useful techniques for student grouping, competency matching, and evolutionary optimization. However, these areas are commonly addressed separately.

Educational team formation research focuses mainly on student attributes, while professional staffing research focuses more directly on competencies and project requirements. Multi-objective optimization research provides methods for exposing trade-offs but does not automatically define which objectives are most appropriate for undergraduate project allocation.

The identified research gap is therefore the limited work addressing undergraduate project allocation where **technical project suitability and student preference are maintained as separate competing objectives and returned as multiple nondominated alternatives for academic decision-making**. The proposed conceptual framework combines student competency indicators and ranked project preferences with approved projects and their technical requirements. Where necessary, a project pool selection stage identifies candidate project subsets. Heuristic-Seeded NSGA-II then optimizes Technical Requirement Deficit and Preference Dissatisfaction and returns nondominated alternatives for academic review.

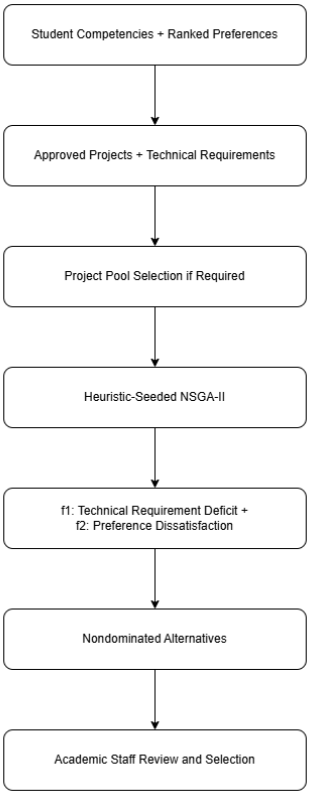


FIGURE 2.1: Conceptual framework of the proposed team project allocation approach

The framework retains academic staff as the final decisionmaker. The optimization process provides structured alternatives, while the final allocation is selected after academic review.

3. RESEARCH METHODOLOGY

3.1. Research design

This research followed a **design and evaluation approach** in which a computational framework was first developed for undergraduate team project allocation and then evaluated using controlled experiments, scalability testing, structural verification, and a real-response case study.

The framework treats team formation as a multi objective optimization problem. Two objectives are minimized separately: **Technical Requirement Deficit** f_1 and **Preference Dissatisfaction** f_2. This allows different tradeoffs between project suitability and student preference to be represented as nondominated alternatives rather than being combined into one fixed weighted score.

The research design consisted of four main stages: input representation, multi-objective optimization, project pool handling where necessary, and evaluation. Controlled synthetic datasets were used to test optimization behavior under different levels of technical versus preference conflict, while a real response case study was used to examine applicability using linked student competency and preference information.

3.2. Data participants or study materials

Three main forms of data were used in the evaluation.

First, controlled synthetic datasets were generated for cohorts of 20, 40, 60, and 100 students under low, medium, and high conflict between technical suitability and project preference. Ten dataset seeds were used for each cohort-size and conflict level combination, resulting in 120 controlled datasets. These datasets enabled repeatable comparison of the proposed method with baseline approaches.

Second, small controlled cases containing 12 students, three projects, and teams of four were used for exact verification. Because this configuration has 34,650 feasible complete allocations, exhaustive enumeration was practical and the true Pareto front could be calculated.

Third, a real response case study was constructed from a single anonymous student survey. Each respondent provided 16 self-reported technology competency ratings and three ranked project preferences. Sixty-six complete and structurally valid responses were received. Sixty responses were selected using simple random sampling without replacement with fixed seed **20260829**, while six responses were retained as reserves. The case study used 15 project options and teams of four.

The technical requirements for the 15 case-study projects specified the required technology, minimum competency threshold, and required number of qualified team members. These requirements were reviewed and approved before evaluation.

3.3. Experimental or study procedure

3.3.1. Problem Representation

Let Srepresent the set of N students and P the set of approved projects. Each student has a self-reported competency rating s_{ik} from 1 to 5 for technology k together with ranked project preferences.

Each project p contains a set of technical requirements Rp. Each requirement specifies a minimum competency level L_{pk} and a required number of qualified members M_{pk}.

For a target team size b, the required number of teams is:

$$\boldsymbol{T = [\frac{N}{b}]}$$

A smaller, remainder team is permitted when the total number of students is not exactly divisible by the target team size.

Every student must be assigned exactly once, every team must receive one selected project, and each selected project can be assigned to only one team. Approved projects may remain unused when the project pool is larger than the number of required teams. If $T > P$, the allocation is considered infeasible. These constraints form the structural basis of the allocation process.

3.3.2. Technical Requirement Deficit

A student is considered qualified for a project requirement when the student's self-reported competency rating meets or exceeds the required threshold.

For project p and technology k:

$$q_{ipk} = \begin{cases} 1, & s_{ik} \geq L_{pk} \\ 0, & \text{otherwise} \end{cases}$$

Let N_{pk} be the number of qualified students in the team assigned to project p Requirement coverage is calculated as:

$$C_{pk} = \min\left(1, \frac{N_{pk}}{M_{pk}}\right)$$

Project coverage is the mean coverage of its requirements:

$$C_p = \frac{1}{|R_p|} \sum_{k \in R_p} C_{pk}$$

The project deficit is:

$$D_p = 1 - C_p$$

The first optimization objective is therefore:

$$f_1 = \frac{1}{|P_s|} \sum_{p \in P_s} D_p$$

Where P_s represents the selected projects.

The value of $f_1 = 0$ represents complete coverage of the **modeled technical requirements**. It does not indicate guaranteed project success or objectively measured technical ability.

3.3.3. Preference Dissatisfaction

For Top 3 project preferences, dissatisfaction is assigned as follows:

Preference position	**Dissatisfaction**
First choice	0

Preference position	Dissatisfaction
Second choice	$1/3$
Third choice	$2/3$
Outside Top 3	1

If $d_i(a(i))$ represents the dissatisfaction of student i for the project assigned to that student, the second objective is:

$$f_2 = \frac{1}{N}\sum_{i \in S} d_i\left(a(i)\right)$$

Both f_1 and f_2 are minimized independently. Preference satisfaction displayed to users can be expressed as $1 - f_2$.

3.3.4. Heuristic-Seeded NSGA-II V3

The optimization process uses **Heuristic-Seeded NSGA-II V3**. One chromosome represents a complete student allocation using a permutation of all students divided into team slots.

The main search configuration is:

Parameter	Configuration
Population size	120
Generations	150
Crossover	Order crossover
Mutation	Cross-team student swap
Objectives	f_1 and f_2
Selection basis	Nondominated sorting and crowding distance

The initial population contains a technical-greedy solution, a preference-greedy solution, nearby swap variants, and random feasible individuals. The heuristic

solutions are used only as initialization anchors; subsequent evolution is governed by the two objectives, nondominated sorting, and crowding distance.

Order crossover is used to preserve the permutation representation [12], while NSGA-II provides the underlying nondominated sorting and diversity-preservation mechanism [11]. The result is a set of nondominated alternatives rather than a single automatically selected allocation.

3.3.5. Surplus Approved Projects

When the number of approved projects equals the number of required teams, all projects are passed directly to the optimizer.

When $|P| > T$, a bounded outer project-subset procedure is used. The default maximum number of candidate subsets is:

$$B = 8$$

If the total number of possible project subsets is eight or fewer, every subset is evaluated. Otherwise, candidate subsets are generated using preference oriented anchors, technical-potential anchors, one-project neighborhood variants, and reproducible random subsets.

V3 is executed separately for each candidate subset. The returned solutions are then pooled and filtered again using Pareto dominance.

Because large surplus-project cases do not evaluate every possible subset, their output is described as a **discovered or approximated nondominated set**, not a guaranteed complete global Pareto front.

3.4. Measures and evaluation metrics

The evaluation used several complementary measures because no single metric can evaluate correctness, optimization quality, scalability, and structural validity simultaneously.

Technical Requirement Deficit f_1 measures the mean uncovered proportion of modeled project requirements. Lower values are better.

Preference Dissatisfaction f_2 measures the average penalty associated with assigned project preferences. Lower values are better.

Hypervolume was used as the principal multi-objective comparison metric. Higher hypervolume indicates better coverage of the objective space by the returned solution set.

Exact Pareto-front recovery was used for the 12 student controlled cases. The output of V3 was compared with the Pareto front obtained through exhaustive enumeration.

Nondominated set size was recorded to examine how the number of alternatives changed as technical-versus-preference conflict increased.

Runtime and objective evaluation count were used to evaluate computational scalability.

Finally, allocation-integrity checks verified that every student was assigned once, team sizes were valid, and selected projects remained unique.

3.5. Data analysis

The proposed method was compared with five alternative approaches: **Technical Greedy, Preference Greedy, Random Search, Weighted Single-Objective GA, and Unseeded NSGA-II**.

For the controlled experiments, stochastic methods were repeated using three optimizer seeds. Dataset level hypervolume results were then used for statistical comparison.

Because the compared results were not assumed to satisfy parametric-test requirements, the Friedman test was used to evaluate overall differences among the stochastic approaches. Kendall's W was used as an effect-size measure.

Where an overall difference was identified, one-sided paired Wilcoxon signed rank **tests** were used for pairwise comparisons between V3 and the alternative stochastic methods. Holm correction was applied to control for multiple comparisons.

The exact 12 student experiments were analyzed separately using exhaustive enumeration rather than statistical approximation. Similarly, the scalability experiment and real-response case study were interpreted according to their individual purposes instead of combining all results into one overall effectiveness claim.

This layered analysis separates evidence of optimization quality, exact correctness, computational scalability, structural validity, and real-response applicability.

3.6. Ethics, privacy, security and responsible AI

The real-response case study used anonymous student responses. Competency and preference information was used only for the purposes of evaluating the team-formation framework.

Competency values are self-reported ordinal indicators. The system therefore does not treat them as verified measures of student ability, and the results should not be used to make unsupported judgments about individual students.

The framework is designed as decision support rather than autonomous decision-making. Academic staff retain authority over the final team project allocation. The optimizer produces alternatives and supporting evidence, while factors outside the mathematical formulation can still be considered before confirmation.

Technical Coverage Inspection is explanatory, and supervisor allocation remains a separate downstream activity. This separation reduces the risk of presenting a mathematical objective as a complete judgment about the suitability or future performance of a student or team.

Only the information required for the defined allocation problem is included in the research formulation. Personal characteristics unrelated to technical requirements and ranked project preferences are not used by the two optimization objectives.

3.7. Reproducibility and research artefacts

Several measures were used to improve reproducibility.

The main V3 configuration was fixed at a population size of 120 and 150 generations. Controlled datasets were generated from recorded dataset seeds, while stochastic methods were evaluated using repeated optimizer seeds.

The real-response sample was selected using the fixed random seed **20260829**, allowing the same 60 responses to be selected again from the 66 valid responses.

For the 12 student verification cases, all 34,650 feasible complete allocations were independently enumerated. This provides a deterministic reference against which the nondominated solutions returned by V3 can be compared.

Research artefacts include the implemented team-formation optimizer, objective-evaluation functions, project-pool selection procedure, synthetic data generators, exact-search verification code, scalability tests, regression tests, and the application services used to integrate the optimization process with IPMS.

These artefacts support traceability between the formal research formulation, the implemented software, and the experiments reported in later chapters.

4. PRODUCT AND SOLUTION DESIGN

4.1. Requirements

The product requirements were derived from the research problem and the need to integrate the individual component with the wider IPMS. The primary user of the component is academic staff responsible for organizing project teams and confirming project allocations.

The main functional requirements are summarized in Table 4.1.

TABLE 4.1: Functional Requirements of the Team Formation Component

ID	Requirement
FR1	Import and validate student, project, requirement and supervisor data.
FR2	Allow staff to configure the required team size.
FR3	Generate feasible team-project allocation alternatives using Heuristic-Seeded NSGA-II V3.
FR4	Display Technical Requirement Deficit and Preference Dissatisfaction for each alternative.
FR5	Allow academic staff to compare and select a nondominated allocation.
FR6	Provide optional requirement-level Technical Coverage Inspection.
FR7	Allocate supervisors after team-project selection.
FR8	Save the staff-confirmed final allocation.
FR9	Maintain the latest confirmed allocation as ACTIVE while retaining previous allocations as ARCHIVED.
FR10	Export confirmed allocations as PDF and Excel reports.
FR11	Support controlled manual revision of a saved allocation.
FR12	Expose the latest official allocation to other IPMS components through REST APIs.

The component also requires several non-functional properties. Input data must be validated before optimization, invalid team or project configurations must be rejected, and confirmed allocations must remain traceable. The user interface should present optimization results in an understandable form rather than exposing internal genetic-algorithm structures. Integration should also remain loosely coupled so that other IPMS components can obtain official team data without directly accessing the Team Formation database.

The final product therefore combines the research optimizer with validation, academic review, persistence, reporting, revision and system-integration functions.

4.2. Component architecture within the complete solution

The Team Formation component follows a layered architecture consisting of a staff facing frontend, a backend application layer and persistent data storage. The backend also provides REST API endpoints that allow other IPMS components to retrieve the latest confirmed team information.

The component begins independently from a staff provided cohort workbook and does not require another IPMS component to execute the initial optimization. After a final allocation has been confirmed, however, it becomes an upstream data provider for the remaining project-management functions.

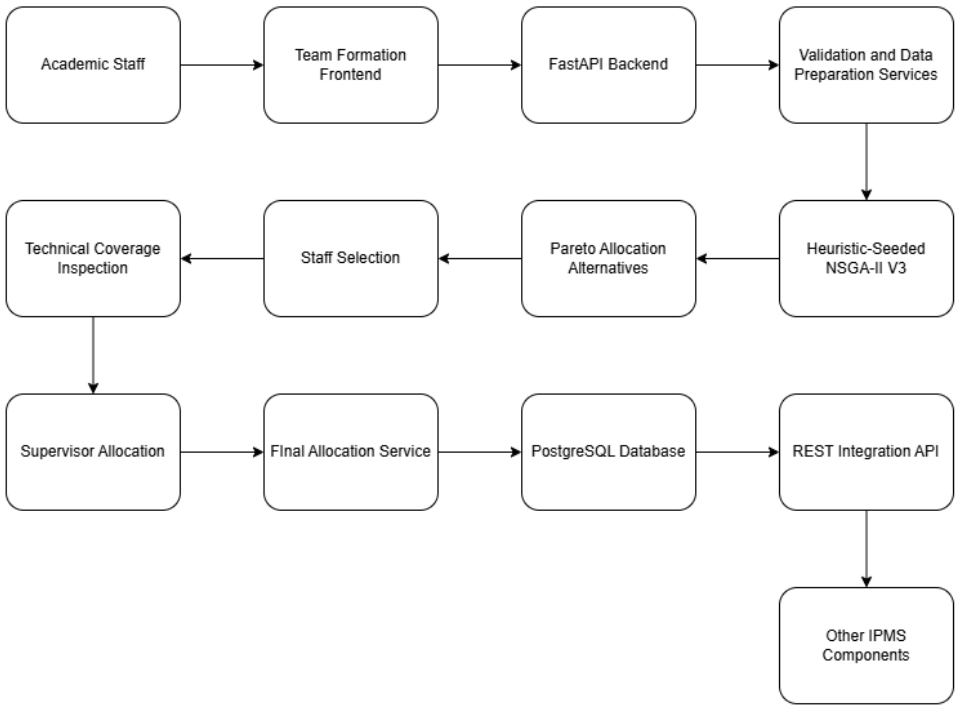


FIGURE 4.1: Architecture of the Intelligent Team Formation and Topic Feasibility component

The frontend provides the academic workflow for setup, validation, generation, review, supervisor allocation and finalization. The backend is responsible for workbook validation, objective evaluation, optimization, technical inspection, supervisor allocation, persistence and integration.

PostgreSQL stores the confirmed allocation information. The newest confirmed allocation is marked as ACTIVE, while earlier confirmed allocations are retained as ARCHIVED. This enables downstream components to request the latest official data without needing to know a particular allocation identifier.

The principal integration endpoint is:

GET /api/final-allocations/active/teams

Supervisor oriented components can also retrieve groups assigned to a particular supervisor using:

GET /api/final-allocations/active/supervisors/{supervisor_id}/groups

Other components consume these APIs rather than accessing the Team Formation database directly. This reduces coupling between components and allows internal implementation details to change without requiring corresponding database changes in other parts of IPMS.

4.3. Design alternatives and decisions

Several important alternatives were considered during the design of the component.

The first decision concerned the optimization model. One option was to combine technical suitability and student preference using a single weighted score. Although this would return one ranked solution, the selected weights would determine the relative importance of the two criteria. The final design therefore retains Technical Requirement Deficit and Preference Dissatisfaction as separate objectives and presents nondominated alternatives to academic staff.

The second decision concerned initialization of the evolutionary search. A completely random initial population provides diversity but does not use useful knowledge available from the problem. The final design therefore combines technical-greedy and preference-greedy seeds with nearby variants and random feasible individuals. This retains population diversity while providing promising starting allocations.

The third decision concerned surplus approved projects. If the number of projects always equals the number of teams would limit practical use. The final design therefore allows approved projects to exceed team demand and introduces a bounded candidate project-subset stage before optimization.

The fourth decision concerned final decision authority. Automatically selecting one Pareto solution would remove potentially relevant academic judgment. The system therefore presents multiple alternatives and allows staff to choose the final allocation. Technical Coverage Inspection is also designed as an explanatory feature rather than an automatic approval mechanism.

A similar separation is maintained for supervisor allocation. Supervisor matching occurs only after the team project allocation has been selected. It uses capacity, expertise, research interest and workload considerations and does not alter the f_1 or f_2 optimization objectives.

Finally, direct database integration between IPMS components was rejected in favor of REST API integration. This design prevents downstream components from depending on the internal Team Formation database schema and provides a clearer ownership boundary for official allocation data.

4.4. Detailed design

The staff workflow is organized into a sequence of controlled stages.

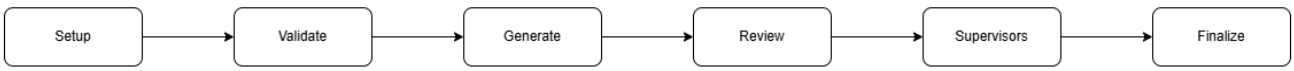


FIGURE 4.2: Staff workflow of the team-formation component

During **Setup**, staff provide the cohort workbook and specify the required team size. The workbook contains the student competency and preference information together with approved project data and related requirements.

During **Validation**, the backend verifies that the required records and values are structurally valid before optimization begins. The number of required teams is also checked against the available approved projects.

During **Generate**, the project pool is prepared and Heuristic-Seeded NSGA-II V3 generates nondominated team-project allocations using Technical Requirement Deficit and Preference Dissatisfaction.

During **Review**, the generated alternatives are presented to academic staff. The purpose is not to label one solution as universally best but to allow comparison of different technical versus preference tradeoffs.

After an allocation has been selected, staff may perform **Technical Coverage Inspection**. This displays information such as the required technology, minimum competency threshold, required member count, qualified students and identified technical gaps. The inspection does not reform teams or rerun the optimizer.

The **Supervisors** stage performs downstream supervisor allocation. Supervisor capacity is treated as a constraint, while expertise, research-interest correspondence and projected workload are used to support matching.

During **Finalize**, the staff-confirmed allocation is stored as the current official allocation. Previously active records are archived rather than deleted.

The product also supports a controlled manual-revision workflow. Staff may export an editable allocation workbook, modify permitted team, project or supervisor assignments and upload it again. The backend validates the revised data and recalculates technical and preference measures rather than trusting values entered in the workbook. Once confirmed, the revision is stored as a new allocation record and becomes the new ACTIVE allocation. Such a record is considered a staff-revised

allocation derived from an optimizer recommendation, not a newly generated Pareto solution.

This approach preserves both human flexibility and traceability.

4.5. Standards and professional practice

Several software engineering and responsible-design practices were considered in the component design.

The integration layer uses HTTP based REST APIs and JSON data exchange. API documentation is exposed through the backend's OpenAPI/Swagger interface, supporting easier integration and testing. Downstream IPMS components communicate through defined service interfaces instead of querying the Team Formation database directly.

Input validation is performed before optimization and again when manually revised allocation data are submitted. This reduces the possibility of malformed cohort information, duplicate assignments or invalid team project configurations entering the final workflow.

Traceability is supported through the ACTIVE and ARCHIVED allocation model. A newly confirmed allocation does not silently overwrite the previous official allocation. This provides a clearer history of academic decisions and allows downstream components to identify the current official data consistently.

The system also follows a human in the loop approach. Optimization results are presented as decision support information rather than autonomous academic decisions. Self-reported competency values are not treated as verified ability, and technical coverage is not presented as a prediction of project success or grades.

The design additionally applies separation of concerns. Team formation, technical coverage inspection, supervisor allocation, persistence, reporting and integration are treated as separate responsibilities. This improves maintainability and prevents operational functions such as supervisor workload from changing the research objectives evaluated by the optimizer.

Overall, the product design converts the multi-objective research method into a practical academic workflow while retaining interpretability, human oversight, system modularity and integration with the wider IPMS.

5. IMPLEMENTATION INTEGRATION AND TESTING

5.1. Development environment and technology stack

The Intelligent Team Formation and Topic Feasibility component was implemented as a web-based application with separate frontend, backend and database layers.

The backend was developed using Python and FastAPI. Pydantic was used for request and data validation, while OpenPyXL was used to process Microsoft Excel workbooks. DEAP supported the evolutionary optimization implementation, and Uvicorn was used to run the FastAPI application.

The staff-facing frontend was implemented using React with Vite. Axios was used for communication with the backend APIs, Recharts supported graphical presentation of selected results, and Tailwind CSS was used for interface styling.

Confirmed allocation information is stored using PostgreSQL, while Microsoft Excel workbooks are used as the main staff-provided input format.

TABLE 5.1: Main Technology Stack

Layer	Technologies
Frontend	React, Vite, Axios, Recharts, Tailwind CSS
Backend	Python, FastAPI, Pydantic, DEAP
File Processing	OpenPyXL, python-multipart
Database	PostgreSQL
API Server	Uvicorn
Input Format	Microsoft Excel (xlsx)

The component was organized using separate routes, services and frontend components so that optimization, validation, feasibility analysis, supervisor allocation and persistence responsibilities remained separated.

5.2. Component implementation

The implemented staff workflow follows six main stages:

Setup → Validate → Generate → Review → Supervisors → Finalize

During the **Setup** stage, academic staff upload the cohort workbook and configure the number of students expected per team. The workbook contains student competency values, project preferences, approved projects, project requirements, project domains and supervisor information.

Before optimization, the backend performs workbook validation. Validation checks include duplicate student identifiers, missing competency values, invalid competency ranges, unknown projects or technologies, invalid project requirements, duplicate preferences, invalid supervisor information and inconsistent team or cohort configurations.

After successful validation, the **Generate** stage invokes the team-formation services. The implementation separates the core responsibilities across services responsible for objective evaluation, NSGA-II optimization, project-pool selection, response preparation and validation.

The main optimization workflow can be summarized as:

Validated Cohort Data

↓

Determine Required Teams

↓

Select Candidate Project Pool if Required

↓

Heuristic-Seeded NSGA-II V3

↓

Evaluate f_1 and f_2

↓

Pareto/Nondominated Alternatives

↓

Return Alternatives to Staff Interface

The frontend presents the generated alternatives during the **Review** stage. Each alternative contains the assigned teams, projects, objective values and supporting technical and preference information. Staff select the allocation they consider appropriate rather than the application automatically selecting one alternative.

After selection, the **Technical Coverage Inspection** feature can be used to examine individual project requirements. It shows the required technology, minimum competency level, required number of members, currently qualified members and identified gaps. This process is explanatory and does not rerun the optimizer or change team membership.

Supervisor assignment is then performed as a separate downstream operation. The implemented supervisor-allocation process considers supervisor capacity as a hard constraint and then uses expertise matching, research-interest matching and projected workload when assigning supervisors.

Finally, the selected team, project and supervisor information is saved as a confirmed final allocation in PostgreSQL.

5.3. Integration with the other components

The Team Formation component provides the initial official team and project structure required by other IPMS components. Integration is performed through **REST APIs** rather than allowing other components to access the Team Formation PostgreSQL database directly.

The main integration architecture is:

Team Formation Frontend
↓
FastAPI Backend
↓
PostgreSQL
↓
REST API
↓
Other IPMS Components

The primary endpoint for retrieving the latest official teams is:

GET /api/final-allocations/active/teams

This endpoint returns the teams, assigned projects, student members and supervisor information belonging to the allocation currently marked as ACTIVE.

A supervisor-specific endpoint is also available:

GET /api/final-allocations/active/supervisors/{supervisor_id}/groups

This allows another IPMS component to retrieve only the groups currently allocated to a particular supervisor.

The use of ACTIVE allocation endpoints means that downstream components do not need to store a fixed allocation identifier. When a new allocation or confirmed revision becomes active, the same API request automatically returns the latest official data.

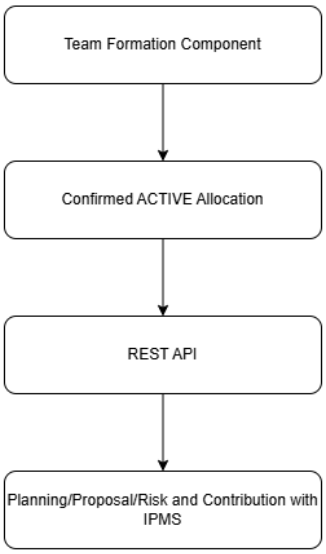


FIGURE 5.1: Integration of the Team Formation component with IPMS

5.4. Verification and software testing

Testing was performed at both the research-algorithm level and the software-product level.

At the algorithm level, exact search verification was used for the controlled 12 student case. Baseline experiments, scalability testing and surplus project regression scenarios were also implemented as separate evaluation services. The detailed

numerical research results are presented in Chapter 6 rather than repeated in this chapter.

At the application level, input validation was tested using invalid and incomplete workbook configurations. The validation layer checks conditions such as duplicate students, competency values outside the accepted range, unknown project references, invalid project requirements and inconsistent team sizes.

Structural allocation checks were also performed. These verify that every student is assigned exactly once, teams contain valid numbers of students, and projects are assigned according to the required uniqueness constraints.

Surplus project behavior was specifically verified using a 60 student and 15 project configuration. Different target team sizes were tested to ensure that the implementation correctly supported unused projects, remainder teams and infeasible configurations.

The controlled manual-revision workflow also contains additional validation. Before a revised allocation can be confirmed, the backend verifies student uniqueness, project validity, team-size rules, supervisor assignments and supervisor capacity. If validation fails, the database is not updated.

The backend recalculates technical coverage, f_1, f_2, preference information and supervisor related information after a revision. Values manually entered the workbook are therefore not trusted as calculated results.

Basic implementation checks were also performed for the completed application, including frontend linting, frontend production build verification and backend Python compilation.

5.5. Deployment and operation

During local development, the FastAPI backend is executed using Uvicorn and exposes its API through port 8003. Interactive Swagger/OpenAPI documentation is available through the backend, supporting endpoint inspection and testing.

The React frontend runs separately using the Vite development server and communicates with the backend through HTTP requests. CORS configuration allows requests from the configured frontend origin.

Once staff complete the workflow, the final allocation is stored in PostgreSQL. Saved allocations follow an **ACTIVE/ARCHIVED lifecycle**. Only one allocation represents the current official state. When a newly confirmed allocation becomes ACTIVE, the previous active allocation is changed to ARCHIVED rather than being deleted.

The system also provides PDF and Excel exports of confirmed allocations. These reports are intended for viewing and record keeping.

A separate editable workbook is provided for controlled manual revision. The revision process is:

Download Editable Allocation

↓

Staff Makes Permitted Changes

↓

Upload Revised Workbook

↓

Backend Validation and Metric Recalculation

↓

Staff Confirmation

↓

Previous ACTIVE → ARCHIVED

↓

Revised Allocation → ACTIVE

A manually changed allocation is stored as a new revision rather than overwriting the original optimizer result. It should therefore be described as a staff-revised allocation derived from an optimizer recommendation, not as a new NSGA-II Pareto solution.

The **/active/...** API endpoints always resolve the latest confirmed version. Consequently, other IPMS components continue using the same integration

endpoints regardless of whether the current official allocation originated directly from the optimizer or from a later staff approved revision.

Overall, the implementation converts the research method into a usable staff workflow while maintaining validation, human confirmation, allocation history and controlled integration with the wider IPMS.

6. RESULTS AND VALIDATION

6.1. Research results

The proposed approach was evaluated using exact Pareto verification, baseline comparison, statistical testing, scalability evaluation, surplus-project constraint testing, and a real-response case study

6.1.1 Exact Pareto Verification

A controlled scenario containing 12 students, three projects, and teams of four was used to verify the optimizer against an exact reference. All 34,650 feasible complete allocations were exhaustively evaluated.

For the medium-conflict scenario, the exact Pareto front was:

$$F_M^* = \{(0,0.20833), (0.05556,0.125), (0.22222,0.08333)\}$$

For the high-conflict scenario, the exact Pareto front was:

$$F_H^* = \{(0,0.375), (0.05556,0.29167), (0.22222,0.20833), (0.38889,0.16667)\}$$

Heuristic-Seeded NSGA-II V3 recovered the complete exact Pareto front in the final repeated medium- and high-conflict verification runs. This confirms that the search procedure could recover the known nondominated solutions in cases where exhaustive verification was computationally possible.

6.1.2 Baseline Comparison

The proposed method was compared with Technical Greedy, Preference Greedy, Random Search, Weighted Single Objective GA, and Unseeded NSGA-II.

TABLE 6.1: Overall Mean Hypervolume by Method

Method	Mean Hypervolume
Heuristic-Seeded NSGA-II V3	**0.9479**
Preference Greedy	0.7793
Unseeded NSGA-II	0.6532

Weighted Single-Objective GA	0.6024
Technical Greedy	0.3721
Random Search	0.3632

V3 achieved the highest overall mean hypervolume. Its value of 0.9479 was approximately 45.1% higher than the 0.6532 obtained by Unseeded NSGA-II. This comparison provides an ablation-style evaluation of heuristic initialization because the underlying NSGA-II framework is retained while the heuristic seeding strategy is removed.

The Friedman test identified a statistically significant overall difference among the stochastic methods:

$$\chi^2(3) = 326.08, \quad p < 0.001$$

Kendall's $W = 0.906$ indicated a large effect. Holm-corrected one sided paired Wilcoxon tests further showed that V3 achieved significantly higher hypervolume than Unseeded NSGA-II, Weighted Single-Objective GA, and Random Search, with $p < 0.001$ for all three comparisons.

The mean number of nondominated alternatives generated by V3 also increased with objective conflict:

- Low conflict: **1.00**
- Medium conflict: **13.21**
- High conflict: **28.78**

This shows that the optimizer returned more trade-off alternatives when technical suitability and preference became increasingly conflicting.

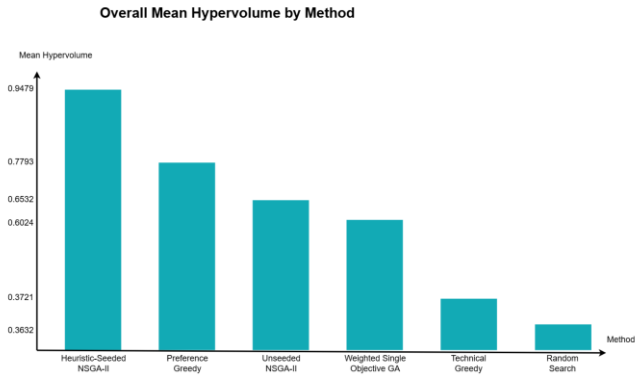


FIGURE 6.1: Overall mean hypervolume of the evaluated allocation methods

6.1.3 Scalability Results

A larger synthetic experiment was conducted using 1000 students, 250 projects, and 250 teams of four.

The optimizer performed approximately 16,744 objective evaluations and returned 25 discovered nondominated alternatives. All returned allocations passed integrity checks for complete student assignment, student uniqueness, valid team sizes, and unique project assignment.

The optimized NSGA-II stage required approximately **28.9 seconds**, while the complete team-formation pipeline required approximately **30.15 seconds**.

An earlier implementation of the same search configuration required approximately 1,336.7 seconds. The optimized implementation therefore achieved an implementation-level speed improvement of approximately **46.3**× (times).

Because this experiment used synthetic student and project data, the result is interpreted as evidence of computational scalability, rather than evidence of institutional deployment performance.

6.1.4 Surplus Project Constraint Results

Structural tests were conducted using 60 students and 15 approved projects with different target team sizes.

TABLE 6.2: Surplus project constraint Verification

Team Size	Required Teams	Result
10	6	Valid; 9 projects remained unused
7	9	Valid; included a remainder team of 4
4	15	Valid equal-count allocation
3	20	Infeasible because only 15 unique projects existed

These results confirmed that the implementation correctly handled unused projects, remainder teams and infeasible configurations. They verify structural behavior of the project-pool extension rather than proving global optimization quality.

6.1.5 Real Response Case Study

The framework was also evaluated using a real-response dataset. A single anonymous survey collected both self-reported competency ratings across 16 technologies and three ranked project preferences from each respondent.

A total of 66 complete and structurally valid responses were obtained. Sixty were selected using simple random sampling without replacement with fixed seed **20260829**, while six were retained as reserves.

The final case study therefore consisted of:

- 60 student profiles;
- 15 approved projects;
- teams of four; and
- linked competency and preference information from the same respondents.

The complete case study scenario and the technical requirements of all 15 projects were reviewed and approved before evaluation. These requirements included the technology, minimum competency threshold and required number of qualified team members.

The optimizer produced:

$$(f_1, f_2) = (0, 0.05)$$

This corresponds to:

- 100% modeled technical requirement coverage, and
- 95% preference satisfaction.

An independent calculation established that $f_2 = 0.05$ was also the minimum achievable preference dissatisfaction under the project-capacity constraints.

Therefore, in this dataset, one feasible allocation simultaneously achieved the minimum technical deficit and the minimum preference dissatisfaction.

Only one nondominated objective point was produced because the two objectives did not require a tradeoff in this dataset. This differs from the deliberately constructed medium- and high conflict datasets, where multiple nondominated alternatives were produced.

6.2. Product and technical results

The research method was implemented as a complete staff facing application rather than being evaluated only as a standalone optimization script.

The implemented system successfully supports workbook validation, team size configuration, multi-objective team formation, alternative comparison, Technical Coverage Inspection, supervisor allocation, persistence, reporting and controlled manual revision.

The validation layer detects invalid data before optimization, including duplicate student identifiers, missing or out-of-range competency values, unknown project references, invalid technical requirements and incompatible team configurations.

Allocation integrity testing confirmed that optimizer outputs preserve complete student assignment, student uniqueness, valid team sizes and unique project assignment.

The completed frontend and backend also passed implementation-level checks. Frontend linting and production build verification completed successfully, while the backend passed Python compilation checks.

The final allocation workflow also supports versioned persistence. A newly confirmed allocation becomes ACTIVE, while the previous active allocation is retained as ARCHIVED. Historical allocations are therefore preserved rather than overwritten.

The controlled manual revision process was also implemented. Revised allocations are validated and their technical coverage, f_1, f_2, preference information and supervisor information are recalculated before confirmation. This prevents manually entered calculated values from being accepted without backend verification.

6.3. User or stakeholder validation

Stakeholder involvement was included in the preparation of the real response case study. The complete case study scenario and the technical requirements for all 15 projects were reviewed and approved before the evaluation. This included the selected technologies, minimum competency thresholds and required qualified-member counts.

This review was important because project requirements directly determine the technical coverage objective. The validation therefore provided academic confirmation that the requirement configurations used in the case study were appropriate for the evaluated projects.

The system also preserves academic authority during normal operation. The optimizer does not automatically impose a final allocation. Instead, staff review the returned nondominated alternatives and select the allocation they consider appropriate.

Technical Coverage Inspection additionally provides requirement level evidence that can be inspected before finalization.

A formal usability study of the staff interface was not conducted as part of the current evaluation. Therefore, this research does not make quantified claims regarding user satisfaction, usability scores, or improvements in academic staff productivity. Such evaluation remains an area for future work.

6.4. Integration results

The component was successfully structured to provide confirmed allocation information to other IPMS components through REST APIs.

The main integration endpoint is:

GET /api/final-allocations/active/teams

This endpoint provides the current official team list, including team numbers, projects, student members and supervisor information.

A supervisor specific endpoint is also available:

GET /api/final-allocations/active/supervisors/{supervisor_id}/groups

This allows consuming components to obtain only the current groups assigned to a particular supervisor.

The integration design was verified together with the ACTIVE and ARCHIVED allocation lifecycle. Downstream components do not need to store a fixed allocation ID because **/active/...** endpoints automatically resolve the latest official allocation.

When a staff revision is confirmed, the parent allocation becomes ARCHIVED, and the new revision becomes ACTIVE. Downstream components can therefore continue using the same endpoint and receive the updated allocation without changes to their logic.

The integration also avoids direct access to the Team Formation PostgreSQL database. Other IPMS components receive only the allocation data required for their operation, keeping internal optimization data and database structures isolated from consuming components.

6.5. Summary of findings

The evaluation produced evidence across several dimensions.

Exact enumeration showed that V3 could recover the known Pareto front in the verified controlled scenarios. Baseline evaluation showed the highest overall mean

hypervolume for V3, with statistically significant improvements over the evaluated stochastic alternatives.

The scalability experiment demonstrated that the optimized implementation could process a synthetic scenario of 1,000 students and 250 projects in approximately 30.15 seconds while maintaining allocation integrity.

Surplus project tests confirmed correct handling of unused projects, remainder teams and infeasible project-supply conditions.

The 60-student real response case study demonstrated application of the framework to linked student competency and preference information. The resulting allocation achieved $f_1 = 0$ and $f_2 = 0.05$, corresponding to complete modeled technical coverage and 95% preference satisfaction for that case.

Finally, product and integration testing confirmed that the research method was incorporated into a functioning staff workflow with validation, persistence, revision, reporting and REST based integration.

The implications of these results, their relationship to the research questions and previous literature, and the limitations of the findings are discussed in Chapter 7.

7. DISCUSSION

7.1. Interpretation of findings

The results indicate that undergraduate team-project allocation can be represented effectively as a multi-objective problem in which technical project suitability and student project preference remain separate. This is important because the two criteria do not always agree. The controlled experiments showed that the number of nondominated alternatives increased from low to high conflict scenarios, demonstrating that the formulation responds to changing levels of conflict rather than always producing the same type of solution.

The exact Pareto verification also provides evidence regarding the correctness of the search approach. Heuristic-Seeded NSGA-II V3 recovered the complete known Pareto fronts in the verified medium- and high-conflict cases. This supports the use of the approach for searching for substantially larger allocation spaces where exhaustive enumeration is impractical.

The baseline comparison further indicates that heuristic initialization improved the quality of the discovered solution sets. V3 achieved a mean hypervolume of 0.9479 compared with 0.6532 for Unseeded NSGA-II, representing an improvement of approximately 45.1%. The statistical analysis also identified significant differences between the evaluated stochastic methods. However, these results demonstrate superiority only within the evaluated configurations and datasets rather than establishing that V3 will outperform every possible optimization method or parameter setting.

The real-response case study produced a different but useful result. The allocation achieved $f_1 = 0$ and $f_2 = 0.05$, meaning that the modeled technical requirements were fully covered while preference satisfaction reached 95%. The independently calculated preference minimum was also $f_2 = 0.05$. Therefore, the single nondominated point in this case does not indicate a failure to generate alternatives; instead, both objective minima could be achieved simultaneously for this dataset.

These findings address the three research questions. **RQ1** is addressed through the interpretable competency to requirement representation. **RQ2** is supported by the observed Pareto fronts and the increasing number of alternatives under greater objective conflict. **RQ3** is addressed through the controlled baseline comparison, seeding comparison, and statistical evaluation.

7.2. Comparison with literature baselines and existing solutions

The findings are consistent with previous research showing that computational methods can support educational team formation. Studies such as Shaikh [1], Løvold et al. [2], Sanchez-Anguix et al. [3], and Liang et al. [4] demonstrate the usefulness of systematic or algorithmic grouping in educational environments.

However, these studies mainly emphasize student or team characteristics. The current research extends this perspective by making the technical requirements of the assigned project part of the allocation model.

This aspect is closer to professional software-staffing research. André et al. [5] and Paredes-Valverde et al. [6] demonstrate the importance of matching competencies with project requirements, while Stylianou and Andreou [7] and Arunachalam et al. [8] show how multi-objective evolutionary methods can be used for personnel selection.

The present work combines these two perspectives within an undergraduate project environment. Student competency information is evaluated relative to explicit project requirements, while ranked project preferences are maintained as a separate objective.

The research also differs from single-objective or weighted approaches because technical suitability and preference are not collapsed into one predetermined score. This is consistent with Pareto-based team formation work such as Casotti and Krohling [10], where multiple competing criteria are represented through nondominated solutions.

Compared with the evaluated baselines, the proposed V3 approach achieved the strongest overall hypervolume. Preference Greedy performed relatively strongly but

emphasizes one side of the allocation problem, while Technical Greedy similarly focuses on technical quality. The weighted GA represents both criteria but requires them to be combined into a predefined scalar objective. The proposed approach instead preserves the two criteria separately and provides several alternatives when genuine tradeoffs exist.

7.3. Research contribution

The first research contribution is an interpretable competency-to-requirement representation. Instead of evaluating students only through general technical strength, each project specifies the technologies it requires, minimum competency thresholds, and the number of qualified members required.

The second contribution is the formulation of Technical Requirement Deficit and Preference Dissatisfaction as independent objectives. This prevents the importance of the two criteria from being hidden inside a fixed weighted score.

The third contribution is the **Heuristic-Seeded NSGA-II V3 approach**. The novelty is not NSGA-II itself, which is an established algorithm [11], but its adaptation to the undergraduate project allocation problem through the proposed representation, objective functions, heuristic initialization and allocation workflow.

The fourth contribution is support for surplus approved projects. The project-pool procedure allows some approved projects to remain unused when the number of projects exceeds the number of student teams. For large project pools, this produces discovered or approximated nondominated alternatives rather than claiming exhaustive global Pareto optimality.

Finally, the evaluation combines exact verification, baseline comparison, statistical testing, scalability analysis, structural testing and real-response data. This provides evidence for different aspects of the framework rather than relying on a single experiment.

7.4. Product contribution

The research method was implemented as a functioning component of IPMS rather than only as an experimental optimization program.

The product provides a staff-oriented workflow beginning with cohort validation and continuing through allocation generation, alternative review, technical inspection, supervisor allocation and final confirmation.

An important product contribution is the preservation of human decision authority. The system does not automatically determine that one Pareto solution is universally best. Academic staff can inspect the alternatives and select the allocation that is appropriate for the cohort.

Technical Coverage Inspection further improves transparency by showing why a particular team satisfies or fails to satisfy individual project requirements. Supervisor allocation is kept as a separate downstream process so that capacity, expertise, research interest and workload do not alter the two research objectives.

The component also supports persistent official allocations, revision history, PDF and Excel reporting, and REST based integration with other IPMS components. The ACTIVE and ARCHIVED model allows the latest approved allocation to be identified while preserving previous versions.

These features convert the research formulation into a practical decision-support workflow suitable for integration with later project-management activities.

7.5. Limitations and threats to validity

Several limitations must be considered when interpreting the findings.

The primary limitation is that student competencies are based on self-reported ordinal ratings. They are useful indicators for threshold-based matching but cannot be interpreted as objectively measured technical ability. Technical Requirement Deficit therefore represents coverage of the modeled requirements rather than a prediction of project performance, grades, productivity or success.

Preference Dissatisfaction also measures only ranked project preference. It does not represent overall motivation, satisfaction, teamwork quality or student engagement.

The real response evaluation used one 60 student case study with 15 projects. Although it demonstrates application to naturally occurring linked competency and

preference responses, it does not establish institution-wide effectiveness. Broader evaluation would require additional cohorts and institutions.

The 1,000 student scalability experiment used synthetic data and equal numbers of teams and projects. It therefore demonstrates computational scalability rather than real institutional deployment of that size.

Another limitation concerns surplus project handling. When the number of possible project subsets exceeds the configured search bound, not every possible subset is evaluated. Consequently, the resulting alternatives cannot be claimed as the complete global Pareto front.

Internal validity was strengthened through recorded dataset seeds, repeated stochastic runs and independent exhaustive enumeration for the 12 student verification cases. However, only three optimizer seeds were used for the stochastic approaches. In addition, the current seeding comparison removes heuristic seeding as a whole and does not separately isolate the contribution of the technical-greedy seed, preference greedy seed and nearby swap variants.

These limitations mean that the results support the evaluated formulation and experimental settings, but broader claims require further replication and validation.

7.6. Implications

The findings have both research and practical implications.

From a research perspective, the results suggest that project-aware undergraduate team formation benefits from preserving competing criteria separately. When technical suitability and preference genuinely conflict, Pareto optimization allows these trade-offs to remain visible rather than hiding them within one weighted value.

From a practical perspective, the framework can support academic staff by reducing the need to manually compare many possible student + team + project combinations. The results are also interpretable because technical coverage can be traced back to specific project technologies, competency thresholds, required member counts and qualifying students.

The approach is particularly useful as decision support rather than automated decision-making. Academic considerations not represented by f_1 or f_2 can still be considered by staff before confirming an allocation.

The surplus project mechanism also increases applicability to real project environments where the number of approved topics does not necessarily equal the number of available student groups. Once an allocation has been confirmed, it can provide a common team-project structure for downstream IPMS functions such as project planning, supervision, proposal analysis and risk monitoring.

Overall, the research indicates that multi objective optimization can provide a transparent and computationally practical foundation for undergraduate team-project allocation while retaining academic oversight.

8. CONCLUSION AND FUTURE DIRECTIONS

8.1. Conclusion

This research investigated the problem of forming undergraduate project teams while considering two potentially competing concerns: whether a team can satisfy the technical requirements of its assigned project and whether students receive projects aligned with their stated preferences.

To address this problem, an intelligent multi-objective framework was developed using **Technical Requirement Deficit** f_1 and **Preference Dissatisfaction** f_2 as separate objectives. Heuristic-Seeded NSGA-II V3 was used to search the allocation space and return nondominated alternatives rather than forcing the two objectives into a single weighted score.

The evaluation provided evidence from several perspectives. Exact enumeration of the 12 student controlled cases showed that the proposed method could recover known Pareto fronts. Across the controlled baseline evaluation, V3 achieved the highest overall mean hypervolume. The scalability experiment demonstrated processing of 1,000 students and 250 projects in approximately 30.15 seconds, while the real-response case study demonstrated application to 60 linked student competency and preference profiles.

The 60-student case study produced $(f_1, f_2) = (0, 0.05)$, corresponding to complete coverage of the modeled technical requirements and 95% preference satisfaction for that dataset. These outcomes should be interpreted within the defined model; they do not imply guaranteed project success, grades, or objectively measured student ability.

Beyond the optimization method, the research was implemented as a staff-oriented component of the wider Intelligent Project Management System. Academic staff remain responsible for selecting and confirming the final allocation, while the system provides optimization results and explanatory evidence to support that decision.

8.2. Achievement of objectives

The objectives established for this research were substantially achieved.

The first objective was to develop an interpretable representation of student competencies and project technical requirements. This was achieved by representing each project requirement using a technology, minimum competency threshold, and required number of qualified team members.

The second objective was to formulate technical suitability and student preference as separate optimization objectives. Technical Requirement Deficit and Preference Dissatisfaction were implemented independently, allowing trade-offs to be represented directly through nondominated alternatives.

The third objective was to develop a Heuristic-Seeded NSGA-II approach for generating team-project allocations. V3 combined technical-greedy and preference-greedy initialization, nearby swap variants, and random feasible individuals while retaining standard Pareto-based evolutionary selection.

The fourth objective was to support practical project allocation conditions, including surplus approved projects. The implemented project pool mechanism supports unused projects, remainder teams, and detection of infeasible configurations.

The final objective was to evaluate the proposed framework through controlled, baseline, scalability, structural, and real-response studies. Exact Pareto verification, statistical baseline comparison, large-scale synthetic testing, surplus-project regression testing, and a 60-student real-response case study were completed.

Overall, the research objectives were achieved within the defined scope of the study.

8.3. Individual research and product contributions

The principal research contribution is a project aware multi-objective formulation for undergraduate team formation that explicitly connects student competency indicators with project technical requirements while preserving student project preference as a separate objective.

A second contribution is the application of **Heuristic-Seeded NSGA-II V3** to this formulation. The contribution is not the invention of NSGA-II itself, but the combination of the project requirement representation, two objective formulation,

heuristic initialization strategy, surplus project handling, and staff oriented decision support process.

The evaluation framework is also an individual contribution. It combines exact search verification, controlled conflict scenarios, baseline comparison, statistical testing, scalability analysis, structural regression testing, and real-response evaluation rather than relying on only one form of evidence.

The product contribution is the implementation of the research as a usable staff-side component of IPMS. The completed component supports workbook validation, team formation, Pareto alternative review, Technical Coverage Inspection, supervisor allocation, allocation persistence, PDF and Excel reporting, controlled manual revision, and REST based integration.

An important characteristic of the implemented product is its human-in-the-loop design. Academic staff retain final allocation authority, while the system provides transparent technical and preference information to support the decision.

8.4. Future directions

Several extensions could strengthen the research and system in future work.

First, the framework should be evaluated using larger and more diverse real cohorts across multiple institutions. This would provide stronger evidence regarding external validity and practical use in different academic environments.

Second, future studies could incorporate competency evidence beyond self-report, such as relevant coursework, assessments, project history, portfolio evidence, or other validated competency indicators. These sources should complement rather than automatically replace student-provided information.

Third, the contribution of the individual heuristic initialization strategies could be investigated separately. The current evaluation compares the complete heuristic-seeded approach with an unseeded NSGA-II configuration, but future ablation studies could independently examine technical-greedy seeding, preference-greedy seeding, and local swap variants.

Further work could also investigate more extensive project-subset optimization when approved projects greatly exceed available teams. This may improve coverage of the possible solution space beyond the current bounded candidate-subset procedure.

Additional optimization criteria could be explored where appropriate, including carefully defined fairness-related considerations. However, such extensions should retain transparency and should avoid introducing hidden criteria that academic staff cannot interpret.

Longitudinal studies could examine relationships between allocation decisions and later project outcomes. Such research would be necessary before making claims about project completion, academic performance, team effectiveness, or other outcomes that are outside the scope of the present technical-coverage model.

Finally, future institutional evaluation should examine staff acceptance, usability, decision time, perceived transparency, and reduction of manual allocation effort. These studies would complement the current algorithmic and technical evaluation and provide stronger evidence regarding practical adoption.

In conclusion, this research demonstrates that project-aware multi-objective optimization can provide a practical and interpretable basis for undergraduate team project allocation. The resulting framework does not replace academic judgment; instead, it provides structured alternatives and supporting evidence that can improve the transparency and consistency of the allocation process.

The future-work directions above follow the finalized research conclusions, including multi-institution evaluation, competency evidence beyond self report, component-level seeding ablations, longitudinal outcomes, broader project-subset and fairness-aware optimization, and staff-oriented usability studies.

REFERENCES

[1] M. K. Shaikh, "How to form a software engineering capstone team?" *Heliyon*, vol. 7, no. 4, Art. no. e06629, 2021.

[2] H. H. Løvold, Y. Lindsjørn, and V. Stray, "Forming and assessing student teams in software engineering courses," in *Agile Processes in Software Engineering and Extreme Programming – Workshops*, vol. 396, pp. 298–306, 2020.

[3] V. Sanchez-Anguix *et al.*, "Comparing computational algorithms for team formation in the classroom: A classroom experience," *Applied Intelligence*, vol. 53, pp. 23883–23904, 2023.

[4] C. Liang, Y. Toyokawa, and H. Ogata, "Optimizing group formation with a mixed genetic algorithm: An empirical study in active reading using marker data," *International Journal of Computer-Supported Collaborative Learning*, vol. 20, pp. 519–548, 2025.

[5] M. André, M. G. Baldoquín, and S. T. Acuña, "Formal model for assigning human resources to teams in software projects," *Information and Software Technology*, vol. 53, no. 3, pp. 259–275, 2011.

[6] M. A. Paredes-Valverde *et al.*, "An ontology-based approach with which to assign human resources to software projects," *Science of Computer Programming*, vol. 156, pp. 90–103, 2018.

[7] C. Stylianou and A. S. Andreou, "A multi-objective genetic algorithm for software development team staffing based on personality types," in *Artificial Intelligence Applications and Innovations*, vol. 381, pp. 37–47, 2012.

[8] A. Arunachalam *et al.*, "Resolving team selection in agile development using NSGA-II algorithm," *CSI Transactions on ICT*, vol. 4, nos. 2–4, pp. 83–86, 2016.

[9] J. Moreno, D. A. Ovalle, and R. M. Vicari, "A genetic algorithm approach for group formation in collaborative learning considering multiple student characteristics," *Computers & Education*, vol. 58, no. 1, pp. 560–569, 2012.

[10] A. L. N. Casotti and R. A. Krohling, “A multi-objective formulation for the team formation problem using Krippendorff's disagreement and sociometric cohesion with Pareto-solutions obtained via evolutionary algorithms,” *Computers & Operations Research*, vol. 161, Art. no. 106444, 2024.

[11] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002

[12] L. Davis, “Applying adaptive algorithms to epistatic domains,” in *Proceedings of the International Joint Conference on Artificial Intelligence*, 1985, pp. 162–164.

APPENDICES

Appendix A Research instruments or extended results

A.1 Student Data Collection Instrument

The real-response case study collected student competency and project-preference information using a single anonymous questionnaire. The same respondent provided both competency ratings and project preferences so that the relationship between these two types of information was preserved.

Students rated their competency in the following 16 technologies:

No.	Technology
1	React
2	HTML/CSS
3	Angular
4	Vue
5	NodeJS
6	Express
7	Java
8	PHP
9	FastAPI
10	MongoDB
11	MySQL
12	PostgreSQL
13	Firebase
14	Python
15	TensorFlow

16	Pandas

The competency scale used in the study was:

Rating Interpretation

1 Little or no practical experience

2 Able to perform basic tasks with guidance

3 Able to independently implement normal project features

4 Strong practical ability and able to troubleshoot complex features

5 Advanced practical ability and able to lead technical work

These values were treated as self-reported ordinal competency indicators rather than objectively measured technical ability.

Each respondent was also asked to identify three ranked project preferences:

- Preference 1 – Most preferred project
- Preference 2 – Second preferred project
- Preference 3 – Third preferred project

A total of 66 complete and structurally valid responses were obtained. Sixty profiles were selected using simple random sampling without replacement using seed 20260829, while six responses were retained as reserves.

A.2 Project Requirement Representation

Each approved project was represented using explicit technical requirements. A technical requirement contained:

- Technology
- Minimum competency level
- Required number of qualified team members

For example, a project requirement of:

Python – Minimum Level 3 – Required Members 2

indicates that at least two members of the assigned team should report a Python competency level of 3 or above for that requirement to be fully covered.

The 60-student real-response case study contained 15 project options with teams of four.

A.3 Additional Evaluation Information

The research evaluation consisted of complementary experiments designed to validate different aspects of the framework:

- exact Pareto verification using 12 students, three projects and teams of four;
- controlled synthetic experiments using cohorts of 20, 40, 60 and 100 students;
- low-, medium and high conflict scenarios;
- comparisons with greedy, random, weighted and unseeded methods;
- scalability testing using 1,000 students and 250 projects;
- surplus project constraint verification; and
- the 60 student real-response case study.

The detailed results and interpretation of these experiments are provided in Chapters 6 and 7. This appendix provides supporting information only and does not replace the main thesis evaluation.

Appendix B Academic integrity and AI use disclosure

B.1 Generative AI Used

Tool: OpenAI ChatGPT

B.2 Purpose of Use

Generative AI assistance was used during the research and thesis preparation process for activities including:

- improving grammar, academic wording and clarity;
- organizing thesis sections according to the required report structure;

- refining explanations of the implemented methodology and system architecture;
- assisting with presentation of tables, headings and technical descriptions;
- reviewing drafts for repetition and consistency;
- supporting debugging and explanation of selected software-development issues during implementation; and
- assisting in transforming author-provided research results and implementation information into clearly structured academic prose.

Generative AI was not treated as an experimental participant, research dataset, source of empirical evidence, or substitute for the research evaluation.

B.3 Representative Use

Representative prompts and workflows included requests such as:

“Rewrite this methodology section using formal academic language without changing the technical meaning.”

“Organize these research results into the Results and Validation chapter using the required thesis structure.”

“Check this explanation of the NSGA-II team-formation workflow for clarity and consistency with the implemented system.”

“Help explain the integration between the Team Formation component and the other IPMS components.”

“Review this chapter and remove unnecessary repetition while preserving the research results.”

B.4 Human Verification and Responsibility

All AI-assisted content was reviewed against the implemented system, research methodology, experimental outputs, source documents, and research paper before inclusion in the thesis.

The research problem, system implementation, experimental configuration, data collection, evaluation results, statistical findings and final conclusions remain the responsibility of the researcher.

Numerical values used in the thesis were checked against the recorded experimental results rather than accepted solely from AI-generated text. Citations and references were also reviewed against the research sources before inclusion.

The researcher remains responsible for the accuracy, originality, interpretation, citations, software, data, analyses and conclusions presented in this thesis.

B.5 Sections Affected

AI assistance contributed primarily to the language, organization and presentation of material in:

- Abstract;
- Chapters 1–8;
- selected tables and figure descriptions;
- reference formatting;
- acknowledgements; and
- supporting documentation.

AI assistance was also used during selected software-development and debugging activities. The final implementation and research outputs were reviewed and tested by the researcher.